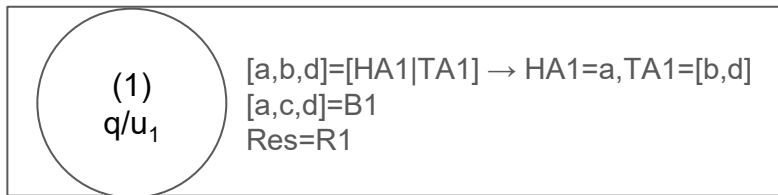


Seminary #2

Union predicate

```
[u1]      union([HA|TA], B, R):-  
[u11]          member(HA, B),  
[u12]          union(TA, B, R).  
[u2]      union([HA|TA], B, [HA|R]):-  
              %not(member(HA,B)),  
[u21]          union(TA, B, R).  
[u3]      union([], B, B).
```

Union predicate



[m₁] member(a, [a,c,d]).

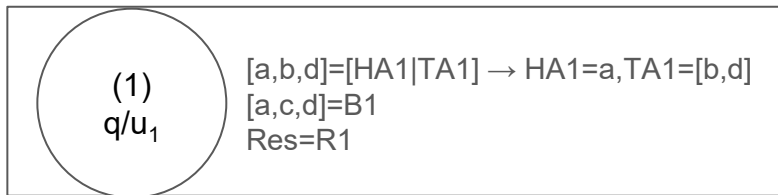
C1: q/head has unified

```
[u1] union([HA|TA], B, R):-  
[u11]      member(HA, B),  
[u12]      union(TA, B, R).  
[u2] union([HA|TA], B, [HA|R]):-  
[u21]      union(TA, B, R).  
[u3] union([], B, B).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]):-  
[m21]      member(X, T).
```

```
[q] ?- union([a,b,d], [a,c,d], Res).
```

Union predicate



[m₁] member(a, [a,c,d]).

C1: q/head has unified

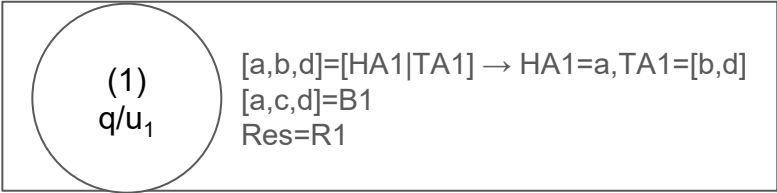
C2: fact? NO

```
[u1] union([HA|TA], B, R):-  
[u11]      member(HA, B),  
[u12]      union(TA, B, R).  
[u2] union([HA|TA], B, [HA|R]):-  
[u21]      union(TA, B, R).  
[u3] union([], B, B).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]):-  
[m21]      member(X, T).
```

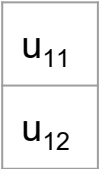
```
[q] ?- union([a,b,d], [a,c,d], Res).
```

Union predicate



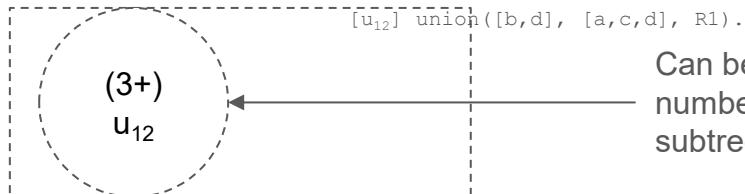
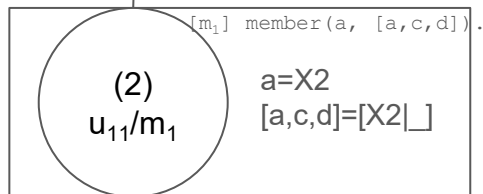
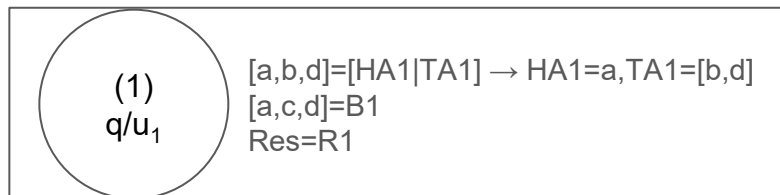
[m₁] member(a, [a,c,d]).

Stack



```
[u1] union([HA|TA], B, R):-  
[u11]      member(HA, B),  
[u12]      union(TA, B, R).  
[u2] union([HA|TA], B, [HA|R]):-  
[u21]      union(TA, B, R).  
[u3] union([], B, B).  
  
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]):-  
[m21]      member(X, T).  
  
[q] ?- union([a,b,d], [a,c,d], Res).
```

Union predicate



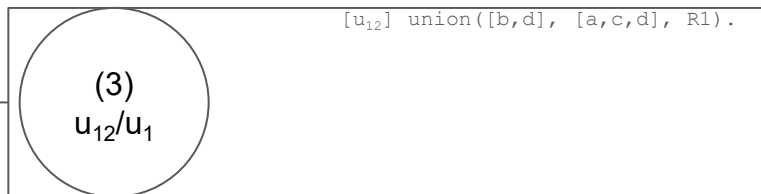
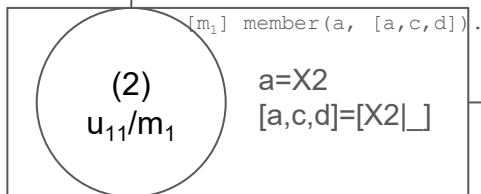
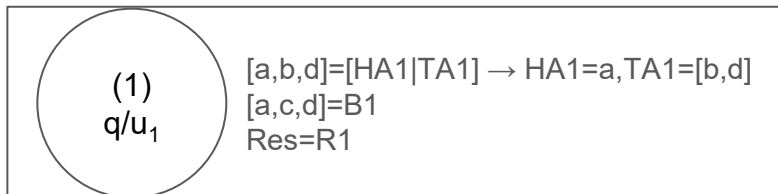
Can be 3, but might be higher node number, depending on 2 (if it has subtree or not)

```
[u1] union([HA|TA], B, R):-
[u11]      member(HA, B),
[u12]      union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]      union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- union([a,b,d], [a,c,d], Res).
```

Union predicate

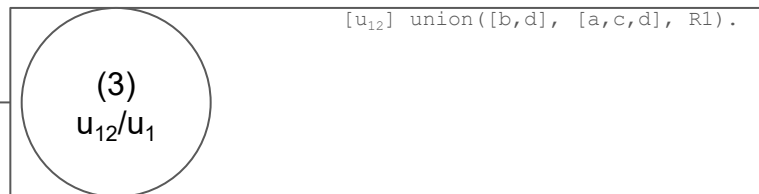
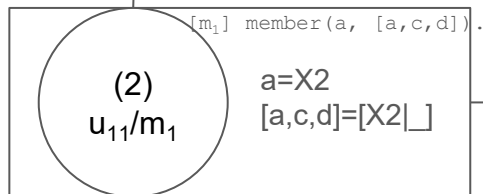
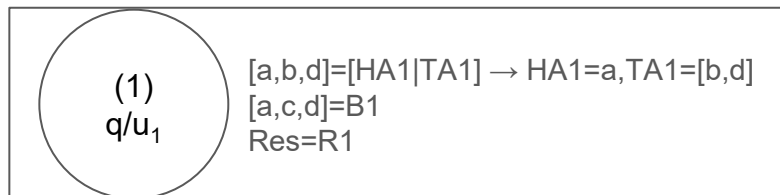


```
[u1] union([HA|TA], B, R):-
[u11]      member(HA, B),
[u12]      union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]      union(TA, B, R).
[u3] union([], B, B).
```

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).
```

```
[q] ?- union([a,b,d], [a,c,d], Res).
```

Union predicate



...

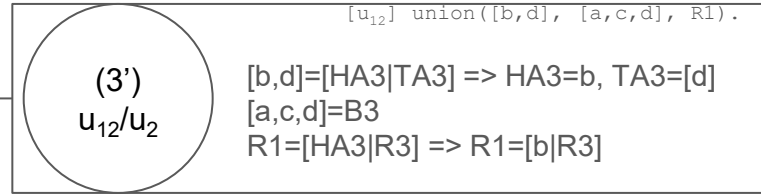
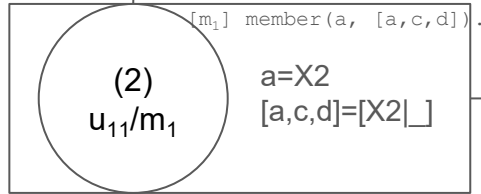
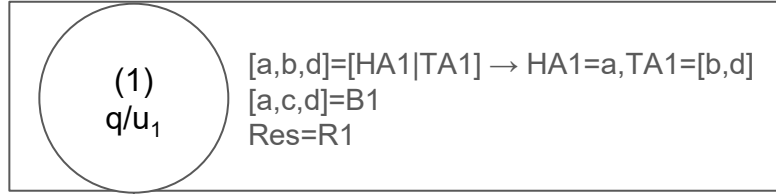
```
[u1] union([HA|TA], B, R):-
[u11]      member(HA, B),
[u12]      union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]      union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- union([a,b,d], [a,c,d], Res).
```

Suggestion: trace it yourselves and see why and how it fails

Union predicate



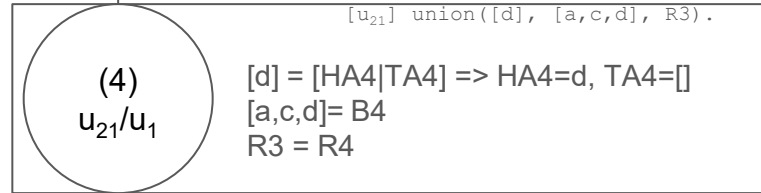
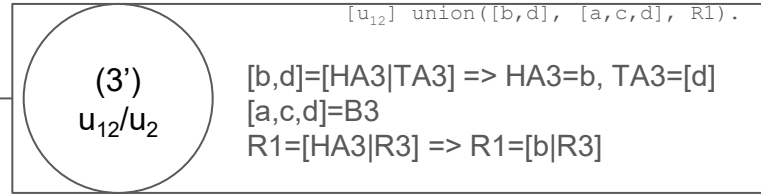
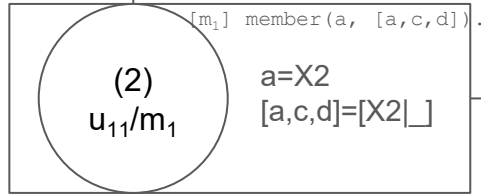
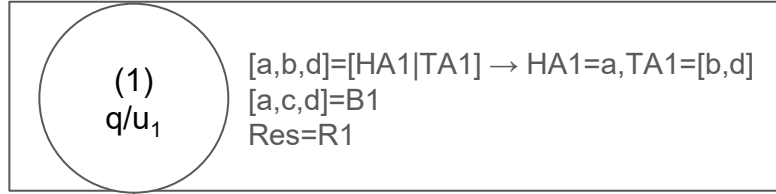
[u₂₁] union([d], [a,c,d], R3).

```
[u1] union([HA|TA], B, R):-
[u11]      member(HA, B),
[u12]      union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]      union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- union([a,b,d], [a,c,d], Res).
```

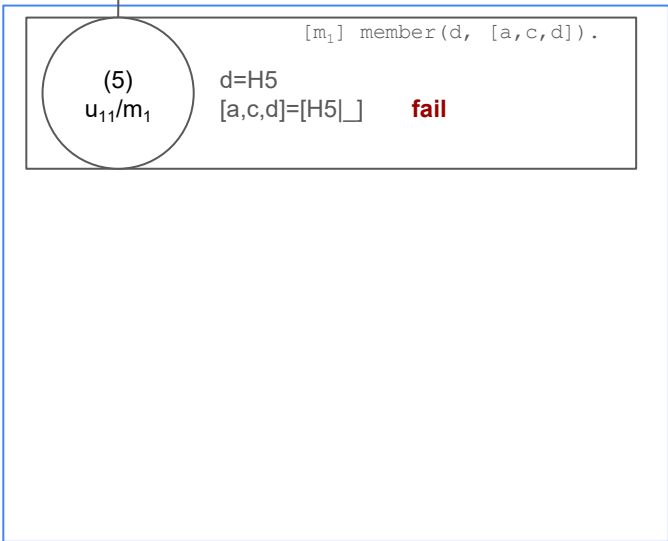
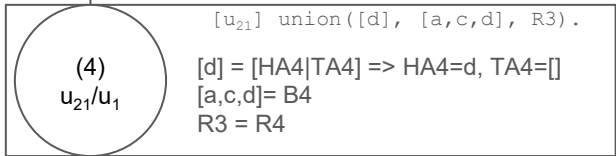
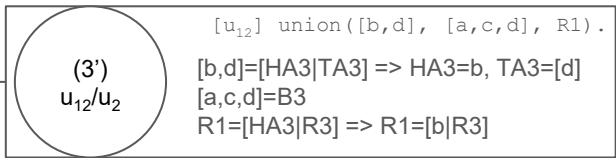
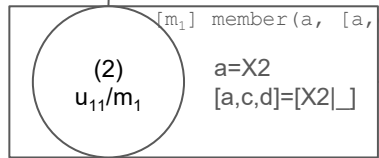
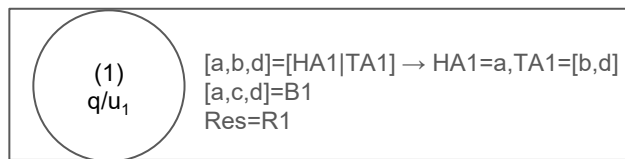
Union predicate



```
[u1] union([HA|TA], B, R):-
[u11]      member(HA, B),
[u12]      union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]      union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- union([a,b,d], [a,c,d], Res).
```



member
trace

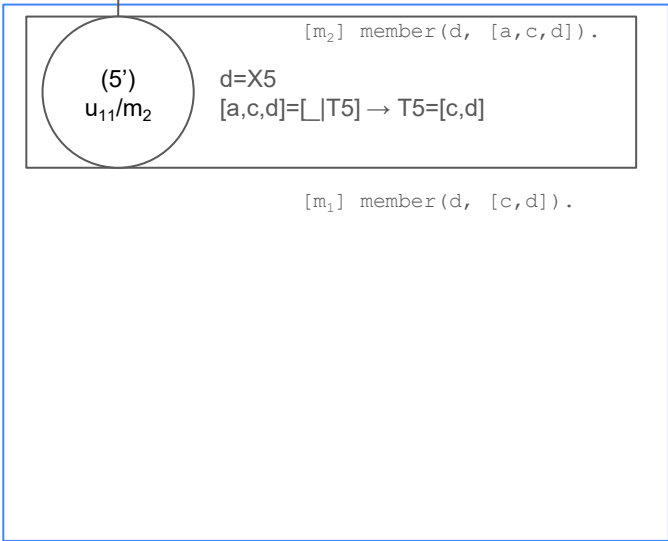
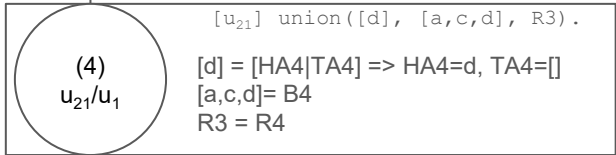
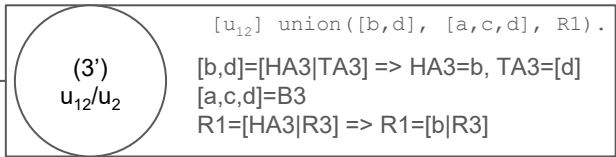
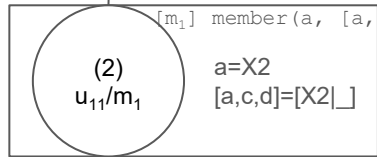
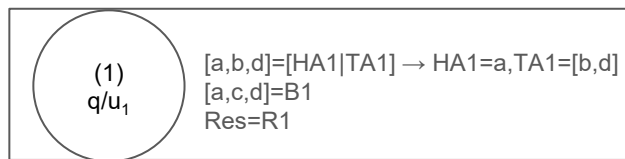
```

[u1] union([HA|TA], B, R):-
[u11]          member(HA, B),
[u12]          union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]          union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]          member(X, T).

[q] ?- union([a,b,d],[a,c,d], Res).

```



```

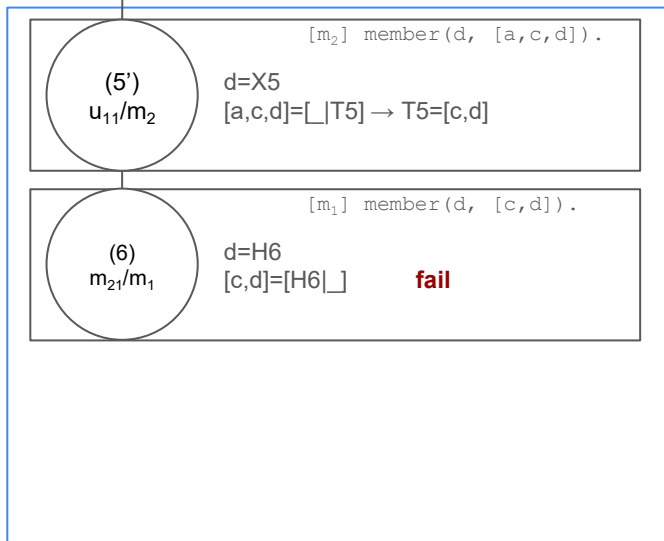
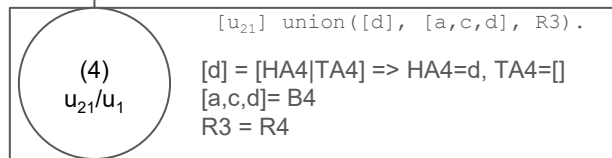
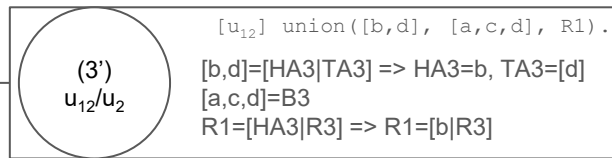
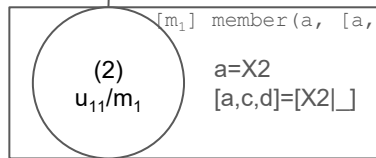
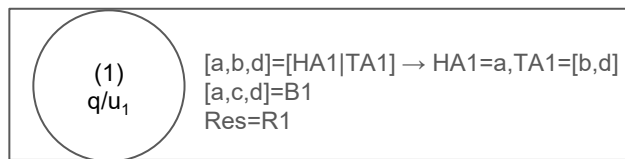
[u1] union([HA|TA], B, R):-
[u11]          member(HA, B),
[u12]          union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]          union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]          member(X, T).

[q] ?- union([a,b,d],[a,c,d], Res).

```

member
trace

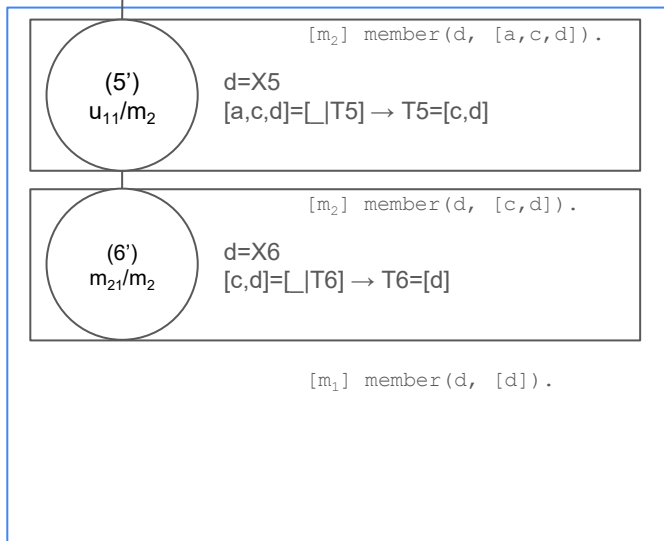
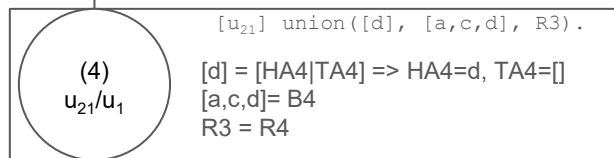
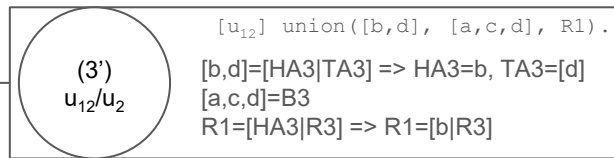
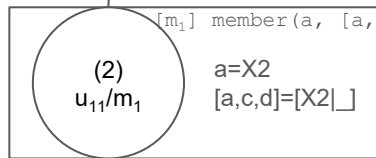
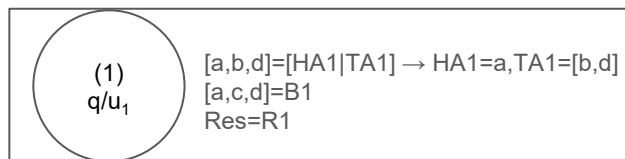


member
trace

```
[u1] union([HA|TA], B, R):-
[u11]          member(HA, B),
[u12]          union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]          union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]          member(X, T).

[q] ?- union([a,b,d], [a,c,d], Res).
```

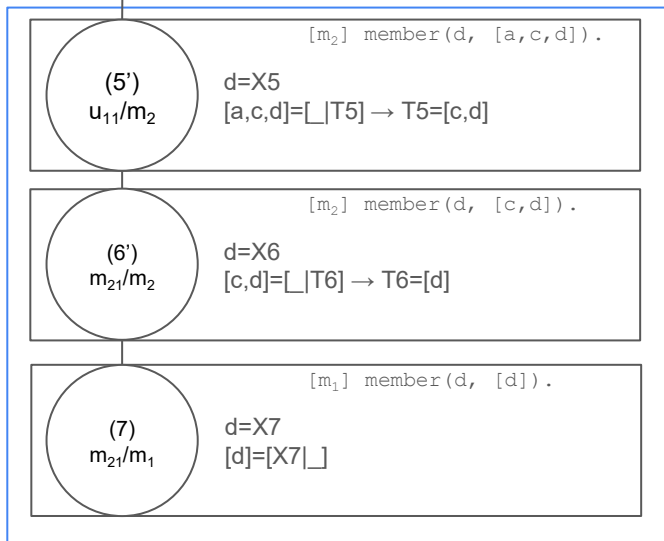
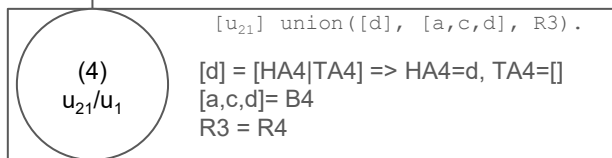
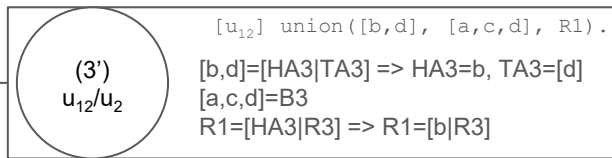
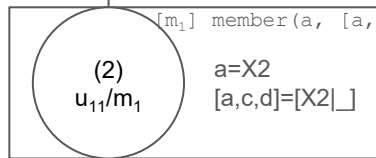
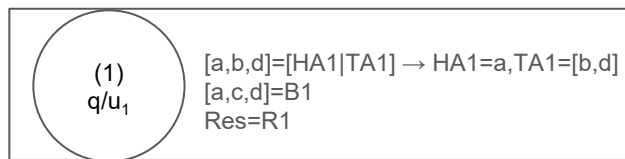


member
trace

```
[u1] union([HA|TA], B, R):-
[u11]          member(HA, B),
[u12]          union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]          union(TA, B, R).
[u3] union([], B, B).
```

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]          member(X, T).
```

```
[q] ?- union([a,b,d], [a,c,d], Res).
```

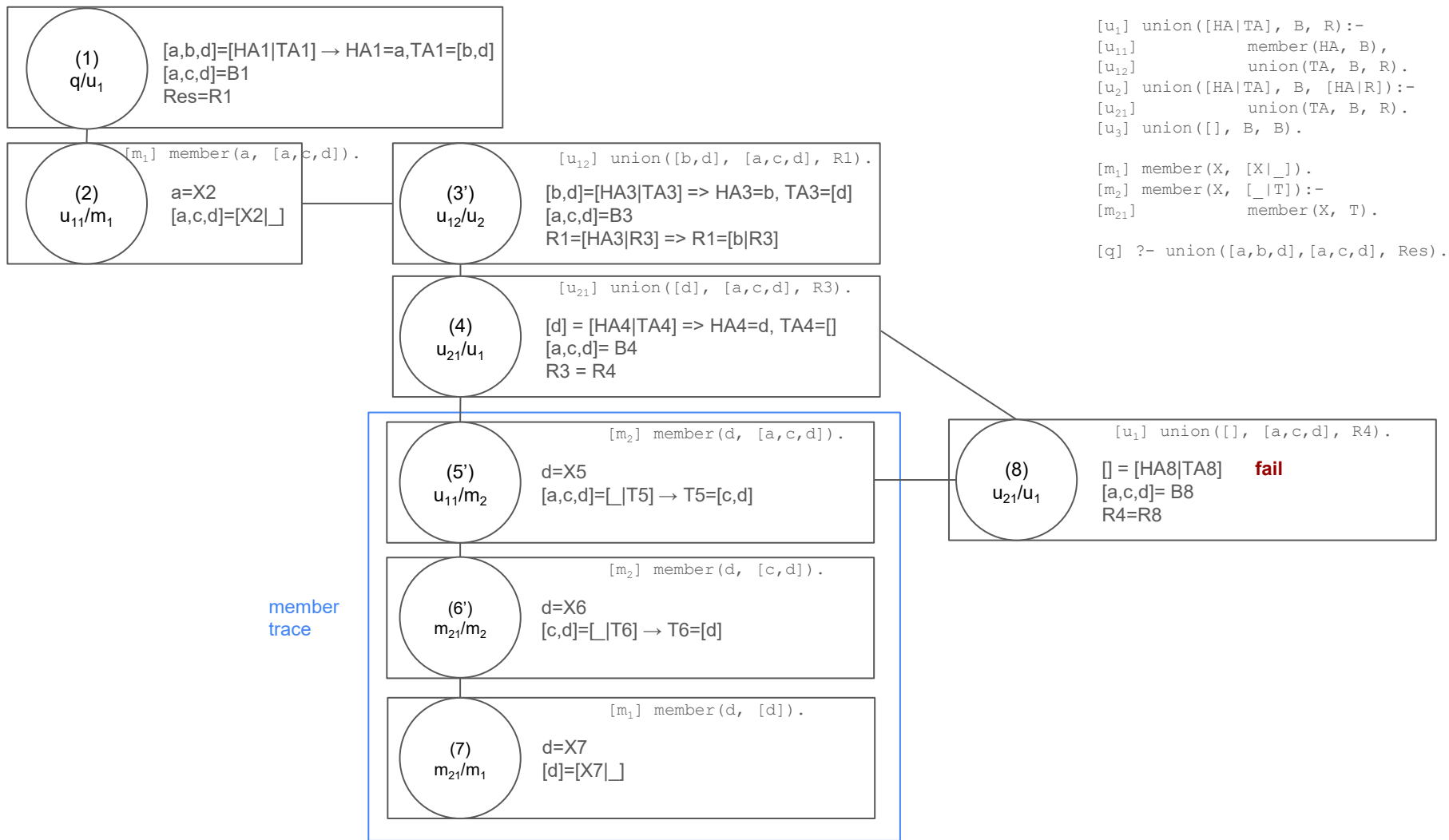


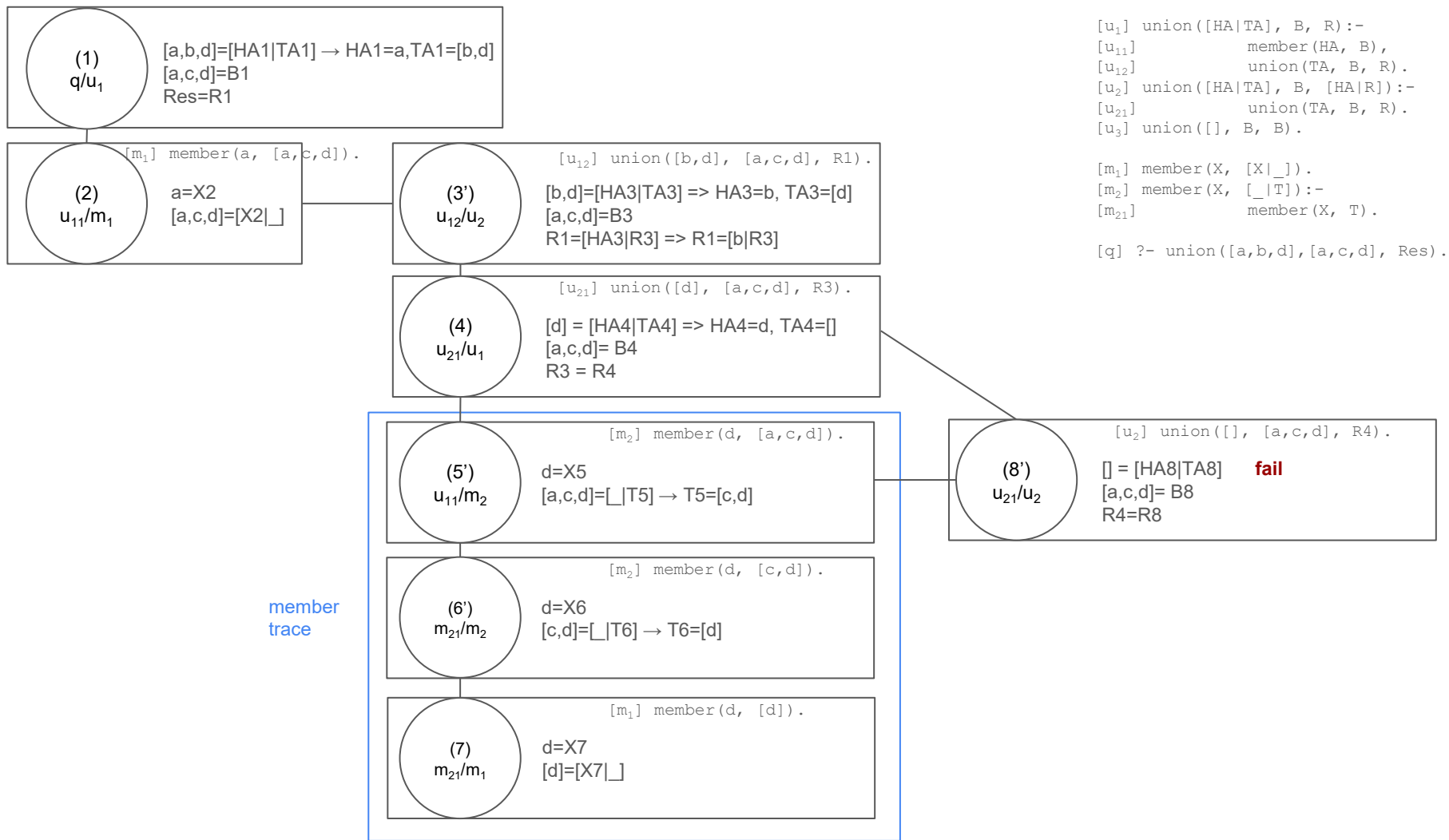
member
trace

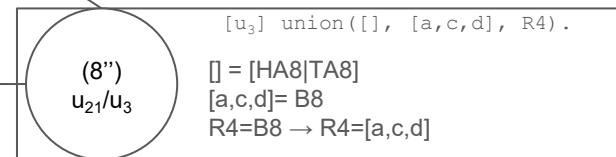
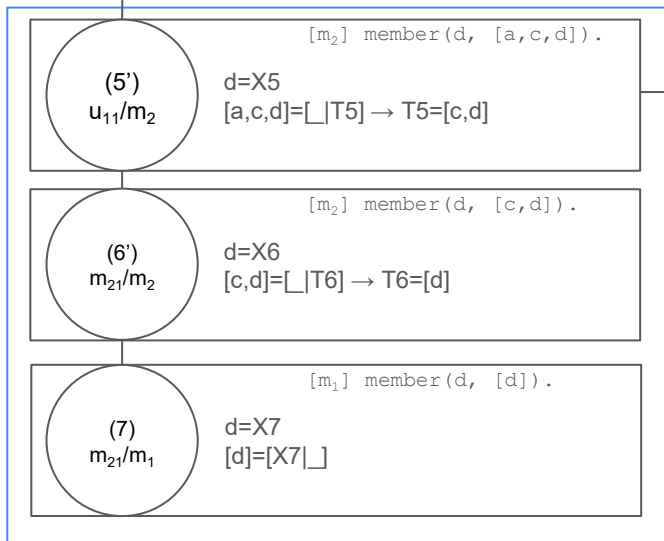
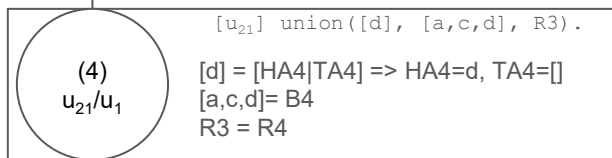
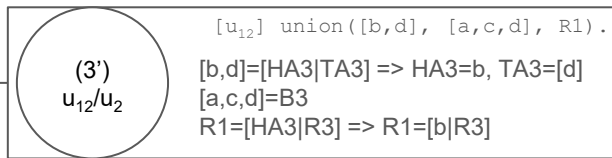
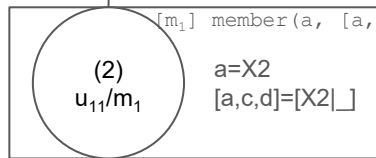
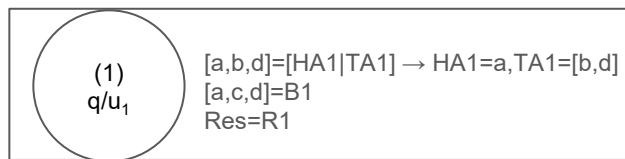
```
[u1] union([HA|TA], B, R):-  
[u11]          member(HA, B),  
[u12]          union(TA, B, R).  
[u2] union([HA|TA], B, [HA|R]):-  
[u21]          union(TA, B, R).  
[u3] union([], B, B).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]):-  
[m21]          member(X, T).
```

```
[q] ?- union([a,b,d], [a,c,d], Res).
```







```
[u1] union([HA|TA], B, R):-
[u11]          member(HA, B),
[u12]          union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]          union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]          member(X, T).

[q] ?- union([a,b,d], [a,c,d], Res).
```

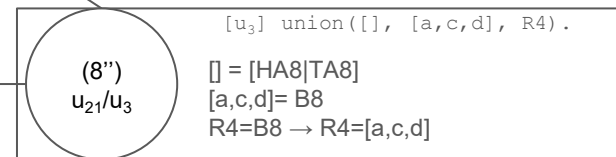
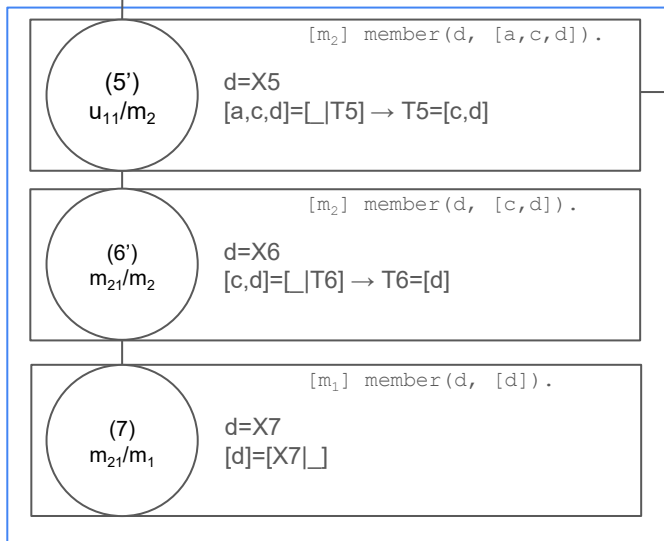
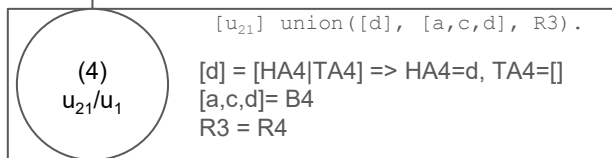
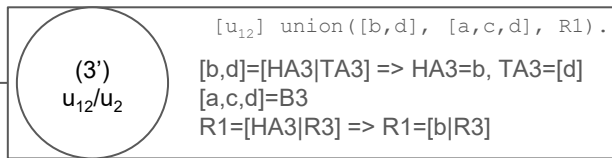
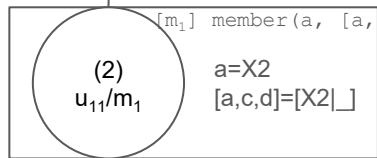
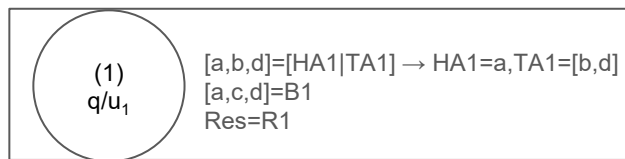
member
trace

(1) (3') (4) (8'')

Res=R1=[b|R3]=[b|R4]=[b|[a,c,d]=[b,a,c,d]

Union predicate - backtracking

- What happens if we repeat the question?
 - It backtracks and we obtain other solutions (ex. $\text{Res}=[b,d,a,c,d]$) because we do not make the *member/2* check in the alternative clause.
 - When backtracking, it will return first to the $\text{union}([d],[a,c,d],R3)$ which will merge with u_2 and will create node **4'**.



```
[u1] union([HA|TA], B, R):-
[u11]          member(HA, B),
[u12]          union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]          union(TA, B, R).
[u3] union([], B, B).

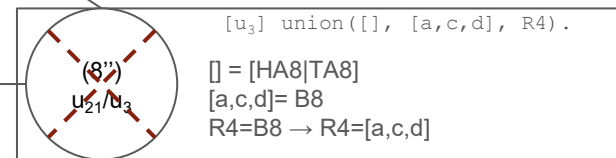
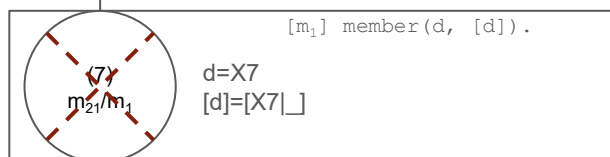
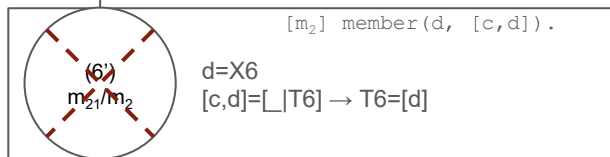
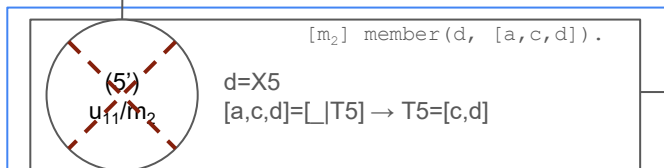
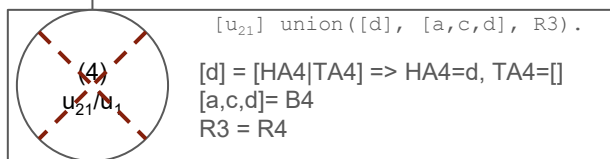
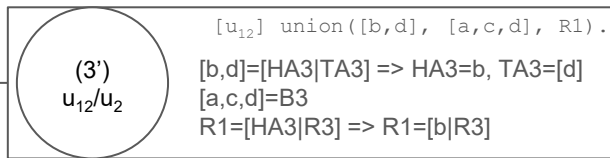
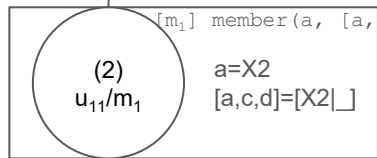
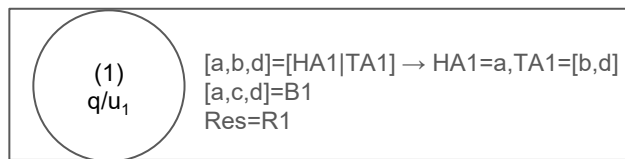
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]          member(X, T).

[q] ?- union([a,b,d], [a,c,d], Res).
Res = [b,a,c,d] ;
```

member
trace

(1) (3') (4) (8'')

Res=R1=[b|R3]=[b|R4]=[b][a,c,d]=[b,a,c,d]
; fail → backtrack



```
[u1] union([HA|TA], B, R):-
[u11]          member(HA, B),
[u12]          union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]          union(TA, B, R).
[u3] union([], B, B).

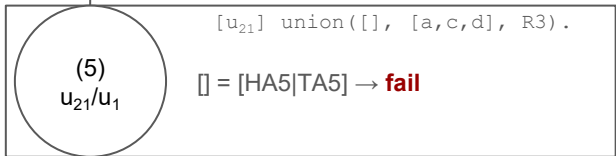
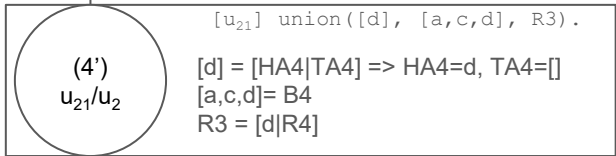
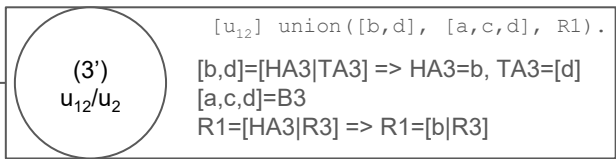
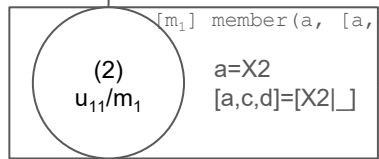
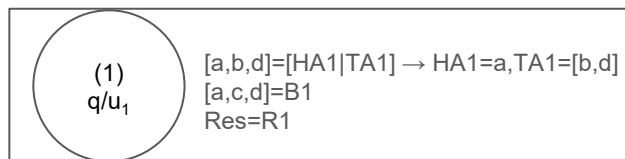
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]          member(X, T).

[q] ?- union([a,b,d], [a,c,d], Res).
Res = [b,a,c,d] ;
```

member
trace

(1) (3') (4) (8'')

Res=R1=[b|R3]=[b|R4]=[b][a,c,d]=[b,a,c,d]
; fail → backtrack



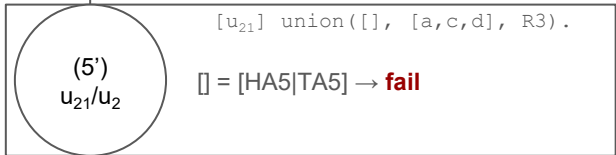
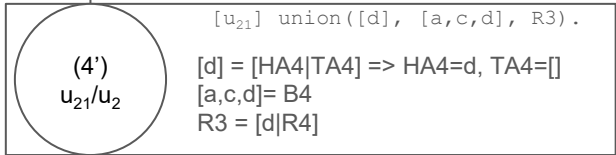
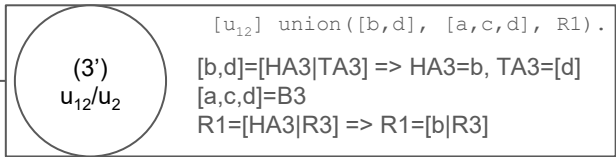
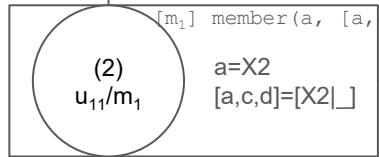
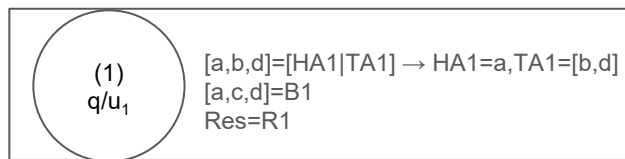
```

[u1] union([HA|TA], B, R):-
[u11]          member(HA, B),
[u12]          union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]          union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]          member(X, T).

[q] ?- union([a,b,d],[a,c,d], Res).
Res = [b,a,c,d] ;

```



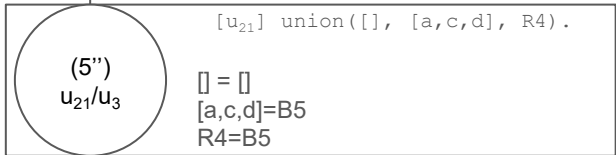
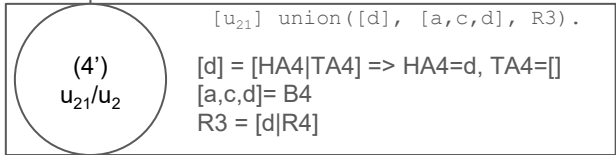
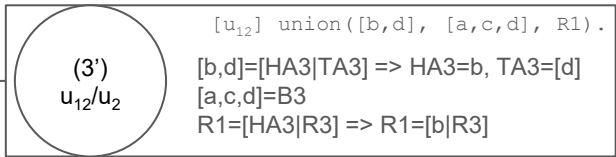
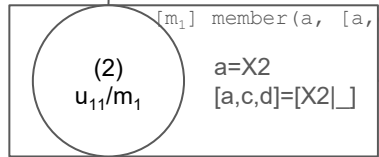
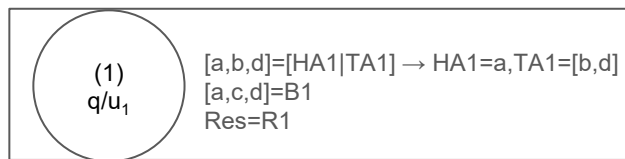
```

[u1] union([HA|TA], B, R):-
[u11]      member(HA, B),
[u12]      union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]      union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- union([a,b,d],[a,c,d], Res).
Res = [b,a,c,d] ;

```



```

[u1] union([HA|TA], B, R):-
[u11]      member(HA, B),
[u12]      union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]      union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- union([a,b,d],[a,c,d], Res).
Res = [b,a,c,d] ;

```

(1) (3') (4') (5'')

Res=R1=[b|R3]=[b|[d|R4]]=[b|[d|[a,c,d]]]=[b,d,a,c,d]

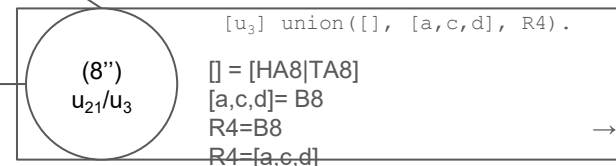
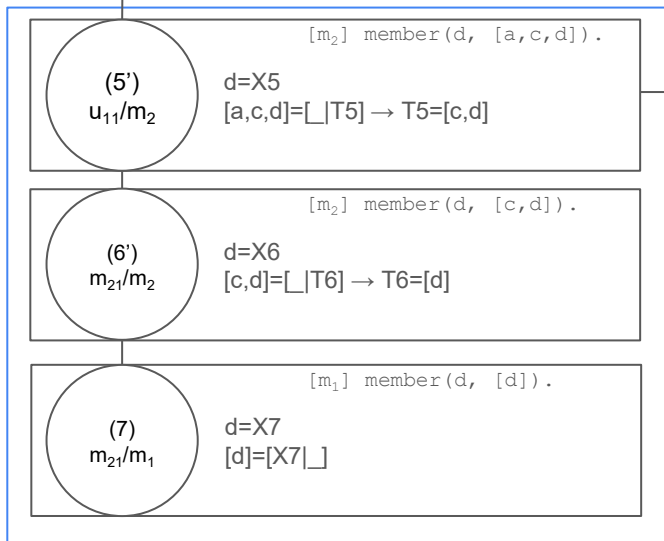
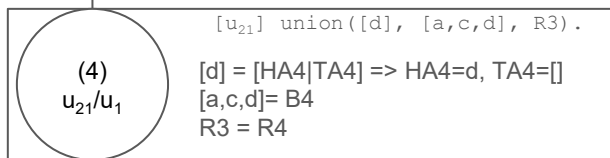
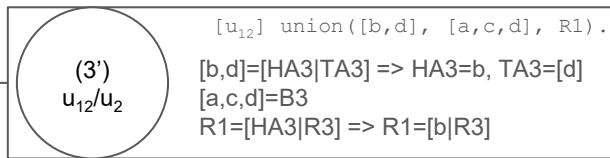
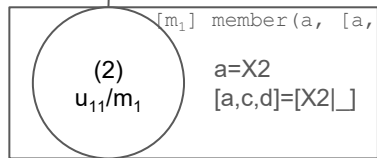
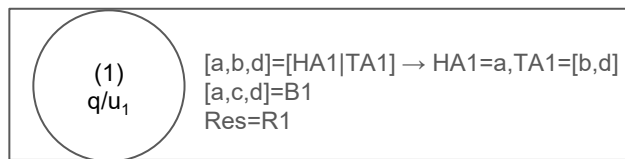
Union predicate - backtracking

- What happens if we repeat the question?
 - It backtracks and we obtain other solutions (ex. $\text{Res}=[b,d,a,c,d]$) because we do not make the *member/2* check in the alternative clause
 - When backtracking, it will return first to the $\text{union}([d],[a,c,d],R3)$ which will merge with u_2 and will create node **4'**.
- All answers:
 - $\text{Res}=[b,a,c,d]$;
 - $\text{Res}=[b,d,a,c,d]$;
 - $\text{Res}=[a,b,a,c,d]$;
 - $\text{Res}=[a,b,d,a,c,d]$;
 - false

Union predicate - the cut

- Adding ! to the predicate

```
[u1]      union([HA|TA], B, R):-  
[u11]          member(HA, B), !,  
[u12]          union(TA, B, R).  
[u2]      union([HA|TA], B, [HA|R]):-  
          % not(member(HA, B)),  
[u21]          union(TA, B, R).  
[u3]      union([], B, B).
```



```
[u1] union([HA|TA], B, R):-
[u11]          member(HA, B),
[u12]          union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]          union(TA, B, R).
[u3] union([], B, B).

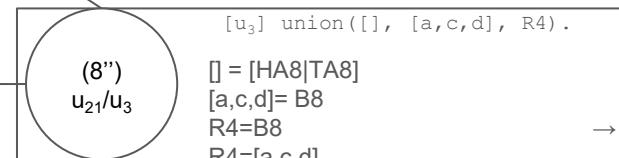
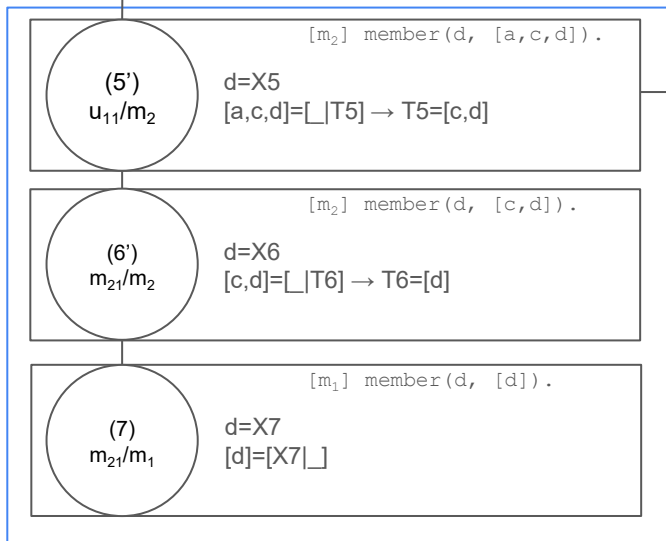
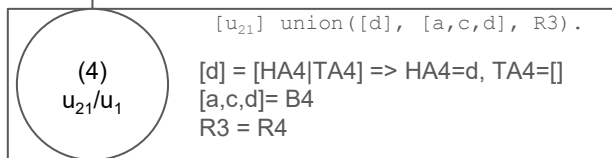
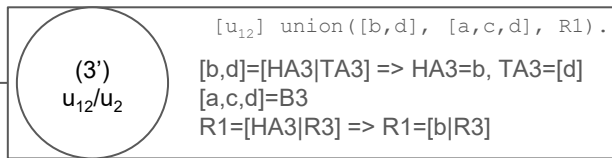
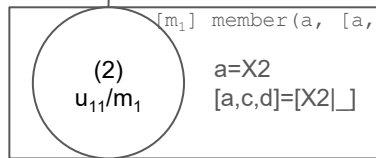
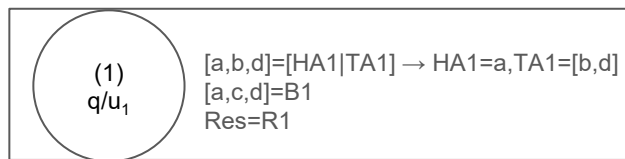
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]          member(X, T).

[q] ?- union([a,b,d], [a,c,d], Res).
```

member
trace

(1) (3') (4) (8'')

Res=R1=[b|R3]=[b|R4]=[b|[a,c,d]=[b,a,c,d]



```
[u1] union([HA|TA], B, R):-
[u11]          member(HA, B),
[u12]          union(TA, B, R).
[u2] union([HA|TA], B, [HA|R]):-
[u21]          union(TA, B, R).
[u3] union([], B, B).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]          member(X, T).

[q] ?- union([a,b,d], [a,c,d], Res).
```

member
trace

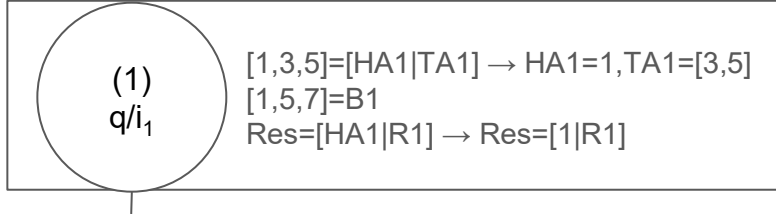
(1) (3') (4) (8'')

Res=R1=[b|R3]=[b|R4]=[b|[a,c,d]=[b,a,c,d]

Intersection predicate

```
[i1]      intersection([HA|TA], B, [HA|R]):-  
[i11]          member(HA, B),  
[i12]          intersection(TA, B, R).  
[i2]      intersection([HA|TA], B, R):-  
              % not(member(HA, B)),  
[i21]          intersection(TA, B, R).  
[i3]      intersection([], _, []).
```

Intersection predicate



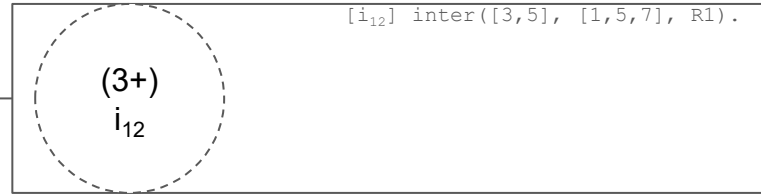
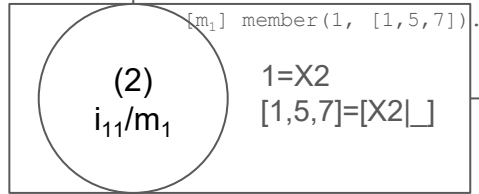
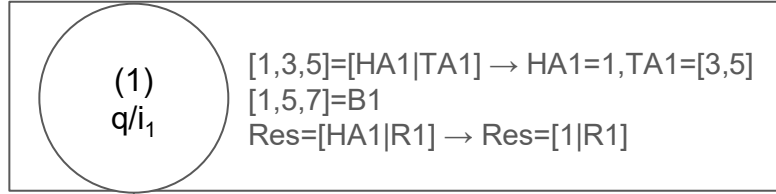
[m₁] member(1, [1,5,7]).

```
[i1] intersection([HA|TA], B, [HA|R]) :-  
[i11]         member(HA, B),  
[i12]         intersection(TA, B, R).  
[i2] intersection([HA|TA], B, R) :-  
[i21]         intersection(TA, B, R).  
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]) :-  
[m21]         member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

Intersection predicate

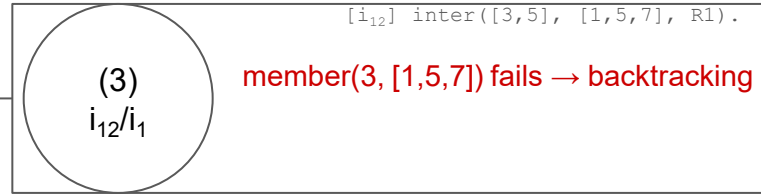
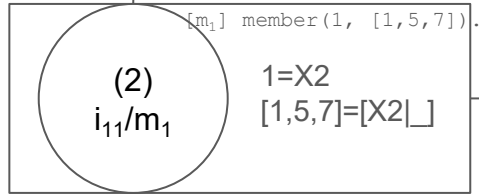
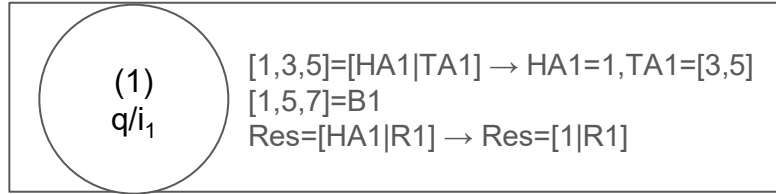


```
[i1] intersection([HA|TA], B, [HA|R]) :-
[i11]      member(HA, B),
[i12]      intersection(TA, B, R).
[i2] intersection([HA|TA], B, R) :-
[i21]      intersection(TA, B, R).
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]      member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

Intersection predicate



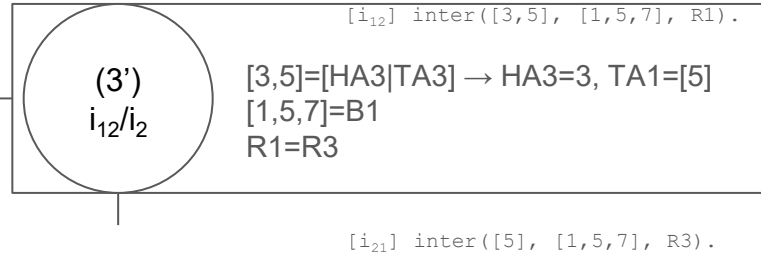
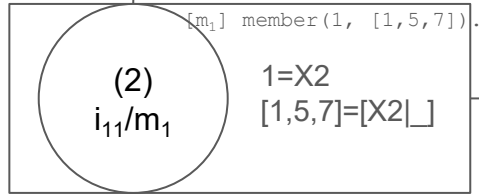
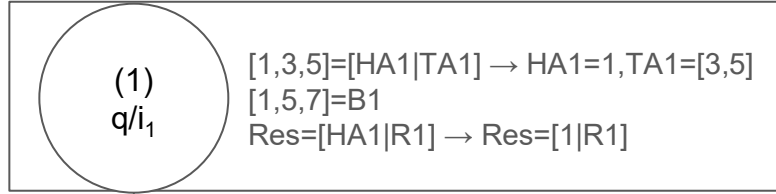
```
[i1] intersection([HA|TA], B, [HA|R]) :-  
[i11] member(HA, B),  
[i12] intersection(TA, B, R).  
[i2] intersection([HA|TA], B, R) :-  
[i21] intersection(TA, B, R).  
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]) :-  
[m21] member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

Suggestion: trace it yourselves and see why and how it fails

Intersection predicate

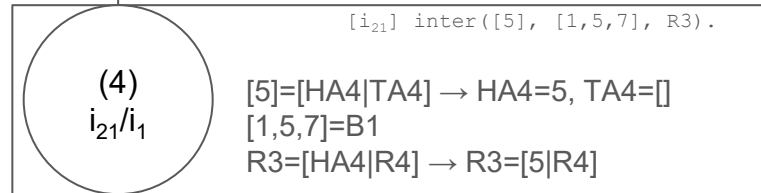
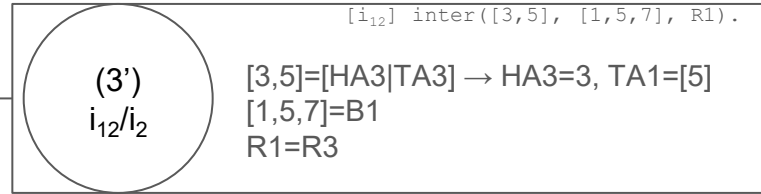
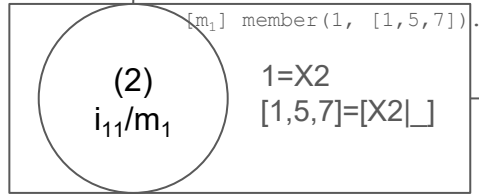
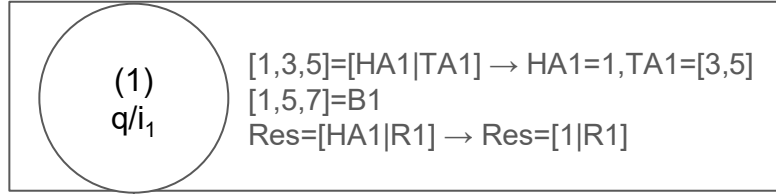


```
[i1] intersection([HA|TA], B, [HA|R]) :-  
[i11]      member(HA, B),  
[i12]      intersection(TA, B, R).  
[i2] intersection([HA|TA], B, R) :-  
[i21]      intersection(TA, B, R).  
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]) :-  
[m21]      member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

Intersection predicate

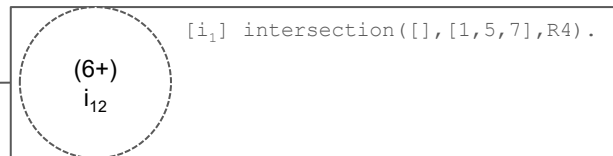
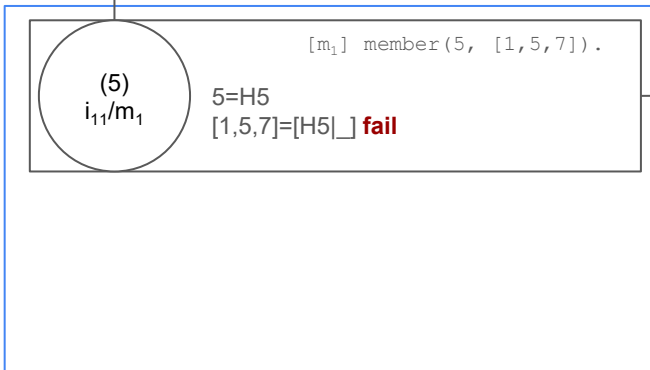
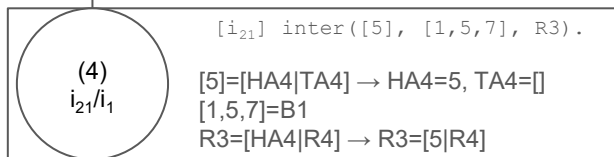
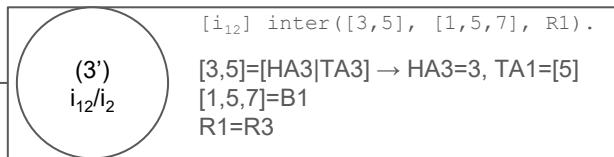
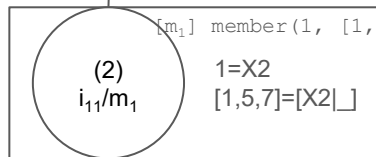
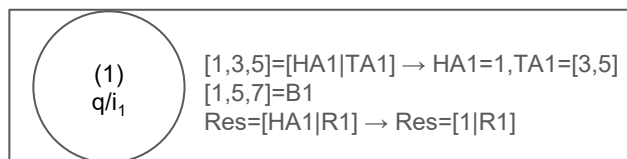


[m₁] member(5, [1,5,7]).

```
[i1] intersection([HA|TA], B, [HA|R]) :-  
    [i11] member(HA, B),  
    [i12] intersection(TA, B, R).  
[i2] intersection([HA|TA], B, R) :-  
    [i21] intersection(TA, B, R).  
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]) :-  
    [m21] member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

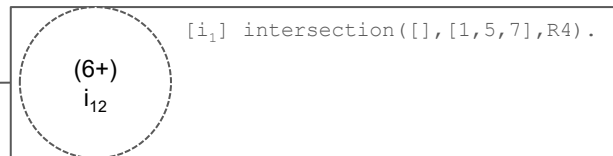
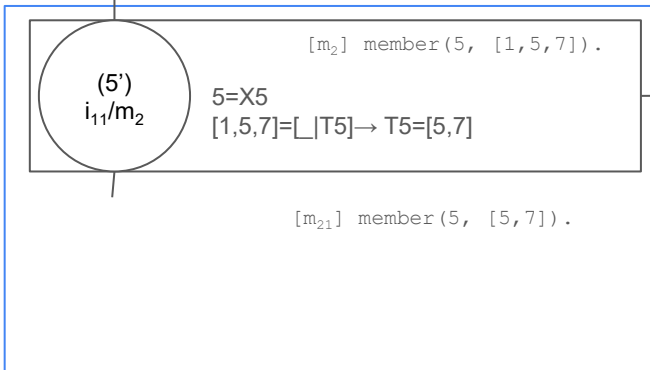
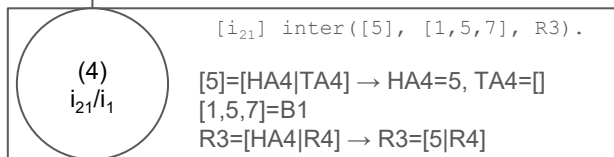
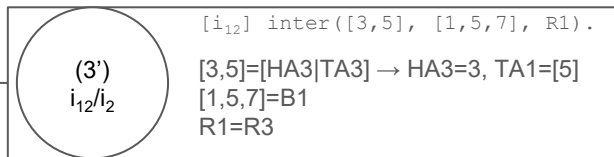
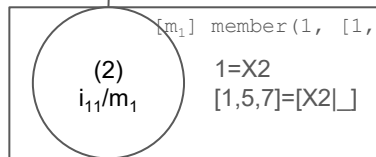
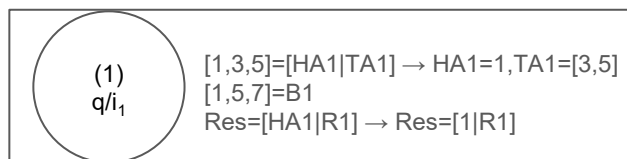


member
trace

```
[i1] intersection([HA|TA], B, [HA|R]) :-  
[i11]      member(HA, B),  
[i12]      intersection(TA, B, R).  
[i2] intersection([HA|TA], B, R) :-  
[i21]      intersection(TA, B, R).  
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]) :-  
[m21]      member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

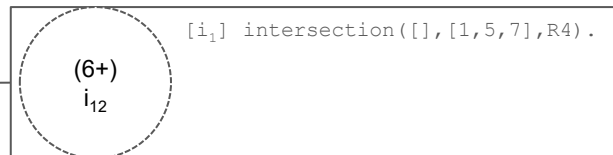
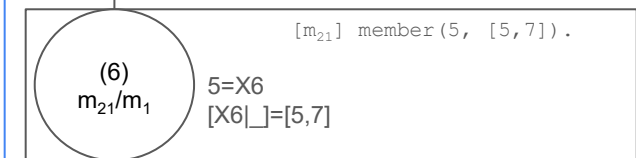
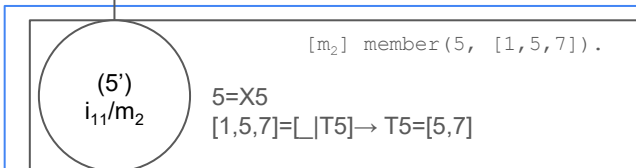
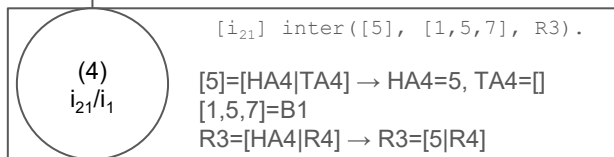
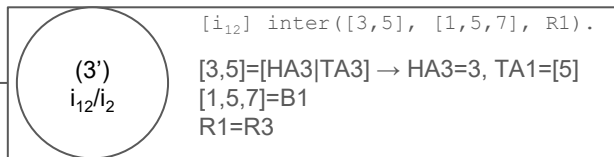
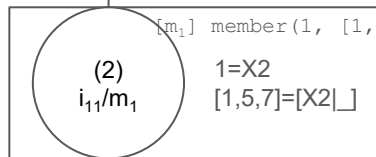
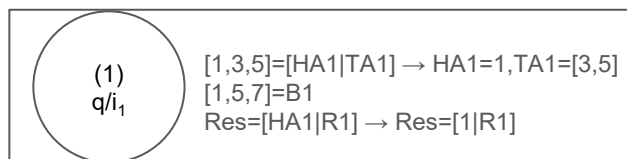


```
[i1] intersection([HA|TA], B, [HA|R]) :-  
[i11]      member(HA, B),  
[i12]      intersection(TA, B, R).  
[i2] intersection([HA|TA], B, R) :-  
[i21]      intersection(TA, B, R).  
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]) :-  
[m21]      member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

member
trace

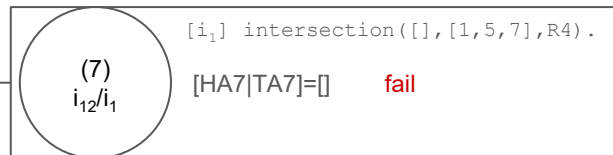
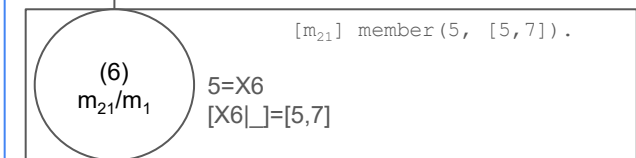
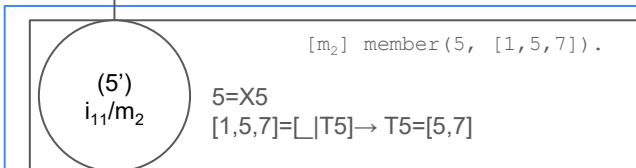
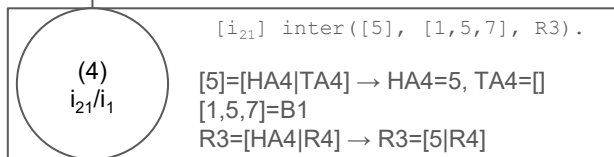
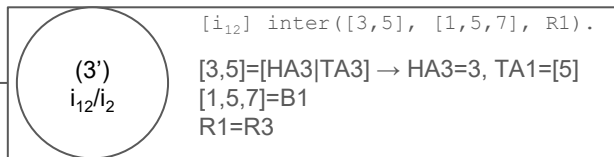
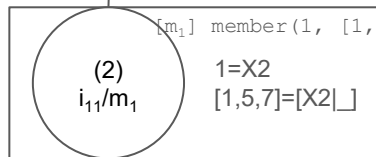
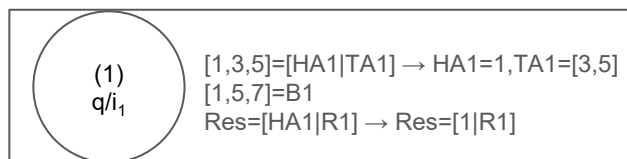


member
trace

```
[i1] intersection([HA|TA], B, [HA|R]) :-  
    [i11] member(HA, B),  
    [i12] intersection(TA, B, R).  
[i2] intersection([HA|TA], B, R) :-  
    [i21] intersection(TA, B, R).  
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]) :-  
    [m21] member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

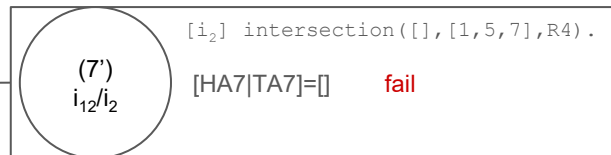
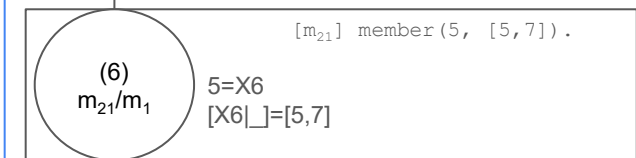
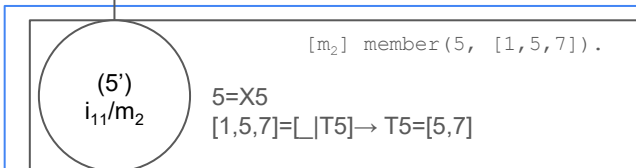
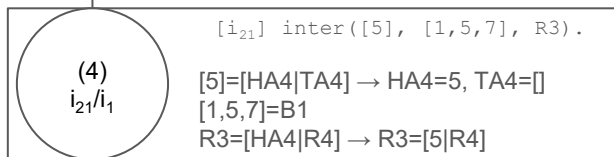
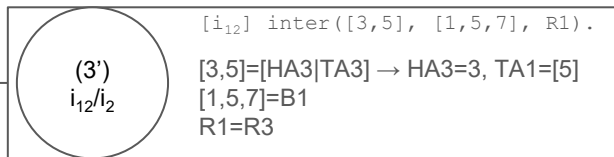
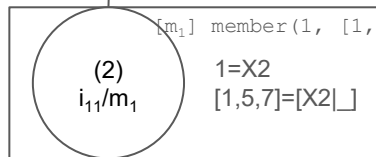
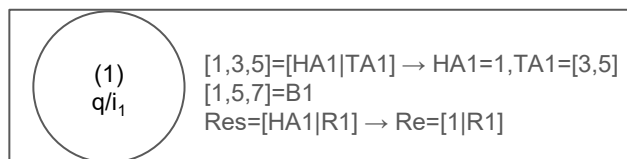


member
trace

```
[i1] intersection([HA|TA], B, [HA|R]) :-
[i11]      member(HA, B),
[i12]      intersection(TA, B, R).
[i2] intersection([HA|TA], B, R) :-
[i21]      intersection(TA, B, R).
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]      member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

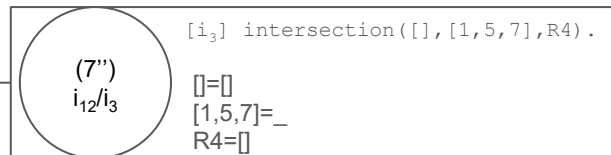
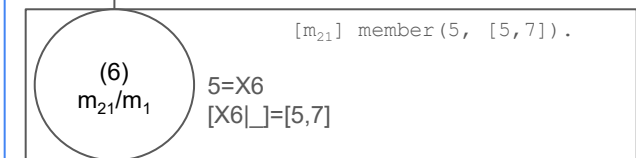
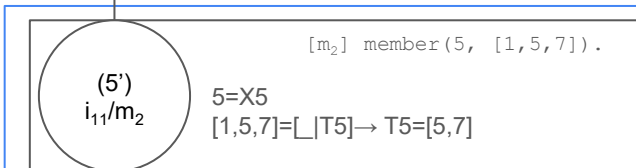
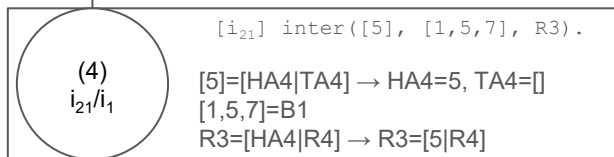
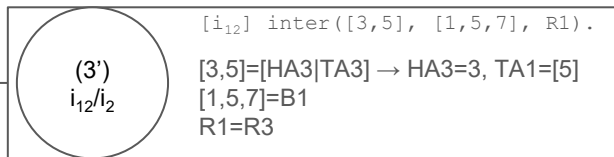
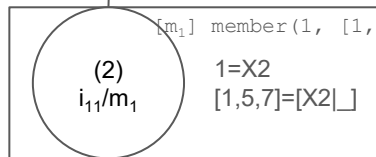
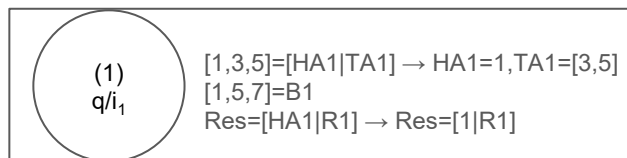


member
trace

```
[i1] intersection([HA|TA], B, [HA|R]) :-  
    [i11] member(HA, B),  
    [i12] intersection(TA, B, R).  
[i2] intersection([HA|TA], B, R) :-  
    [i21] intersection(TA, B, R).  
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]) :-  
    [m21] member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

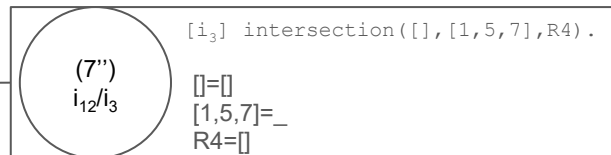
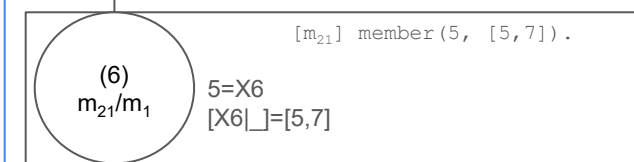
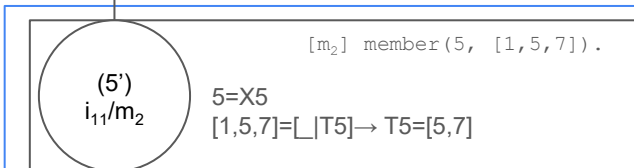
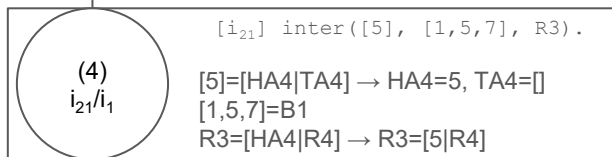
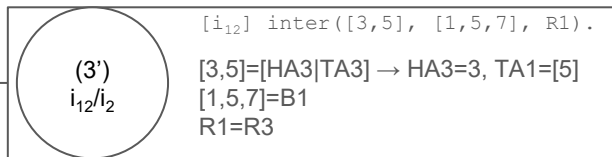
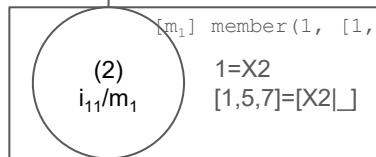
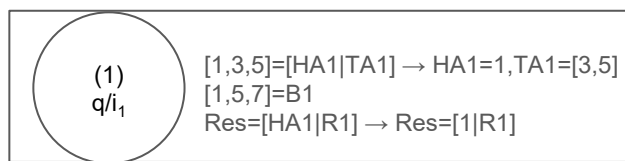


member
trace

```
[i1] intersection([HA|TA], B, [HA|R]) :-
[i11]      member(HA, B),
[i12]      intersection(TA, B, R).
[i2] intersection([HA|TA], B, R) :-
[i21]      intersection(TA, B, R).
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]      member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

```
[i1] intersection([HA|TA], B, [HA|R]) :-  
    [i11] member(HA, B),  
    [i12] intersection(TA, B, R).  
[i2] intersection([HA|TA], B, R) :-  
    [i21] intersection(TA, B, R).  
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]) :-  
    [m21] member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

member
trace

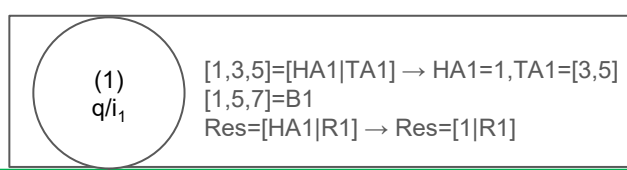
(1)

(3')

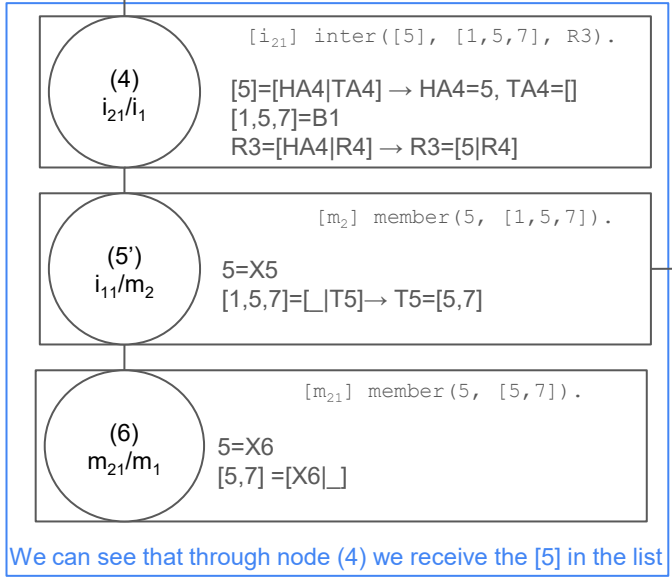
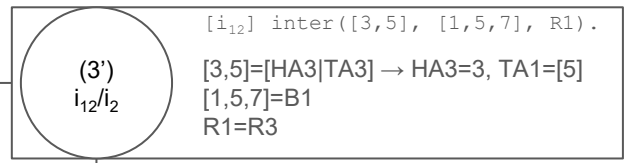
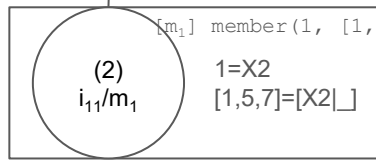
(4)

(7'')

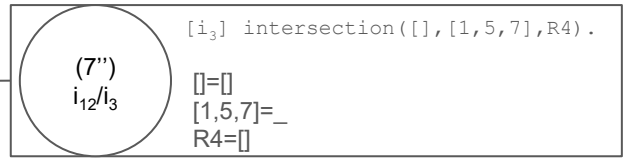
Res = [1|R1] = [1|R3] = [1|[5|R4]] = [1|[5|[]]] = [1,5]



We can see that through node (1) we receive [1] in the list



We can see that through node (4) we receive the [5] in the list



```
[i1] intersection([HA|TA], B, [HA|R]) :-
[i11]      member(HA, B),
[i12]      intersection(TA, B, R).
[i2] intersection([HA|TA], B, R) :-
[i21]      intersection(TA, B, R).
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).
[m2] member(X, [_ |T]) :-
[m21]      member(X, T).
```

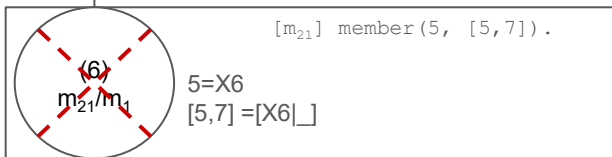
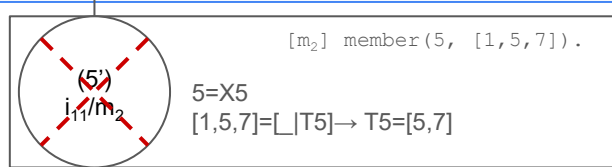
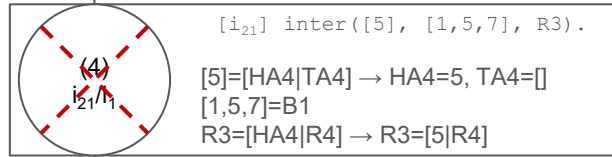
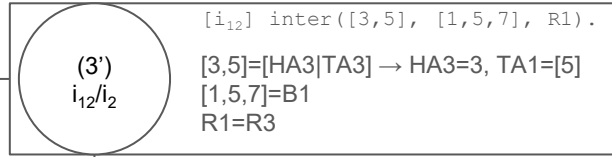
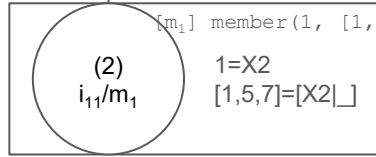
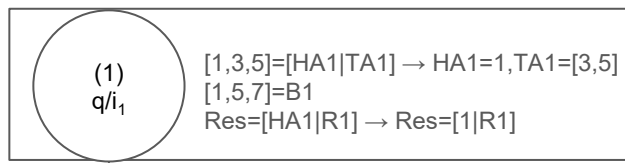
```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

(1) (3') (4) (7'')

$$\text{Res} = [1|R1] = [1|R3] = [1|[5|R4]] = [1|[5|[]]] = [1,5]$$

Intersection predicate - backtracking

- Upon repeating the question, we reach through backtracking node 4 which unified with i_1
 - Now it unifies with i_2



```
[i1] intersection([HA|TA], B, [HA|R]) :-  
    [i11] member(HA, B),  
    [i12] intersection(TA, B, R).  
[i2] intersection([HA|TA], B, R) :-  
    [i21] intersection(TA, B, R).  
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).  
[m2] member(X, [_|T]) :-  
    [m21] member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).  
Res = [1,5] ;
```



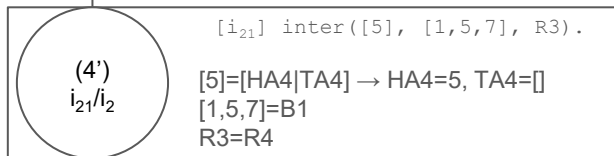
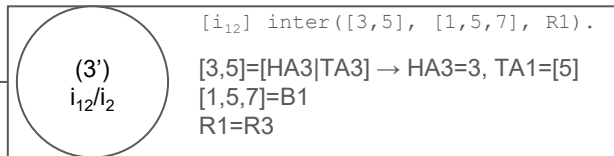
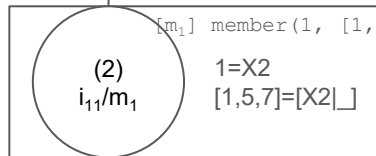
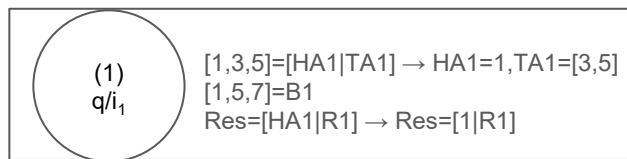
Backtrack starts at (7''), tries to find another way to go but there is no i₄ to go to and it fails

Backtrack then goes to (6), forces (6') from m₂₁/m₂ and this creates new nodes that fail themselves propagating the fail until (4') is created with i₂₁/i₂

member trace

(1) (3') (4) (7'')

Res = [1|R1] = [1|R3] = [1|[5|R4]] = [1|[5|[]]] = [1,5]
; Next → backtrack



$[i_{21}] \text{ inter}([], [1,5,7], R3).$

```

[i1] intersection([HA|TA], B, [HA|R]) :-
[i11]         member(HA, B),
[i12]         intersection(TA, B, R).
[i2] intersection([HA|TA], B, R) :-
[i21]         intersection(TA, B, R).
[i3] intersection([], _, []).

```

```

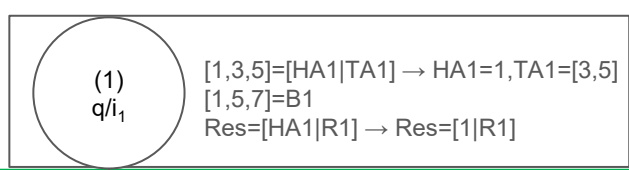
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]         member(X, T).

```

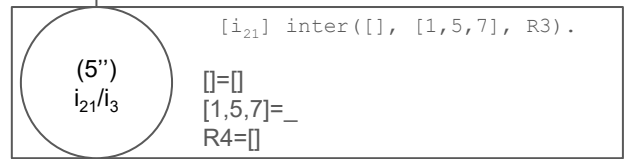
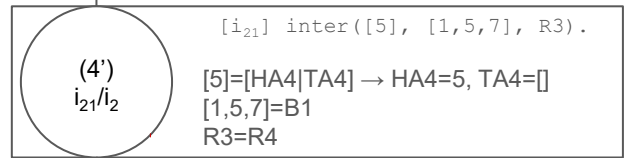
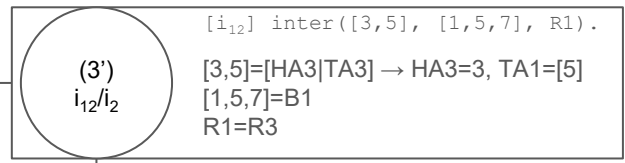
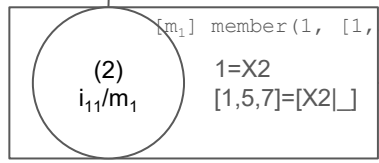
```

[q] ?- intersection([1,3,5], [1,5,7], Res).
Res = [1,5] ;

```



We can see that
through node (1)
we receive [1] in
the list



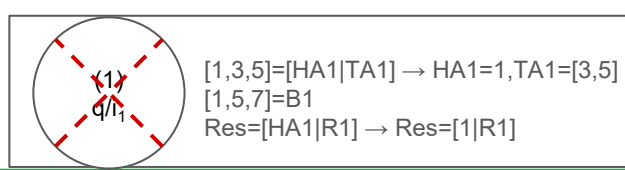
```
[i1] intersection([HA|TA], B, [HA|R]) :-
[i11]      member(HA, B),
[i12]      intersection(TA, B, R).
[i2] intersection([HA|TA], B, R) :-
[i21]      intersection(TA, B, R).
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]      member(X, T).
```

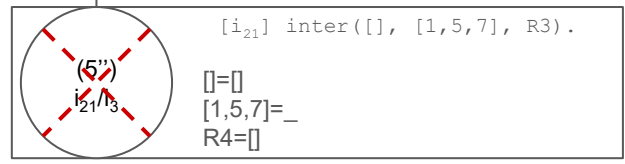
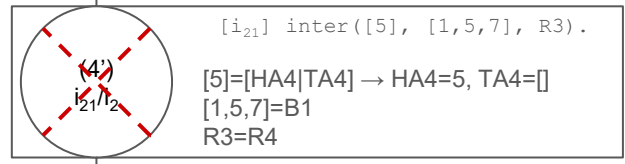
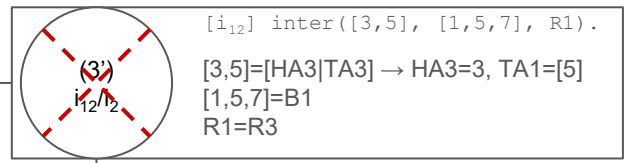
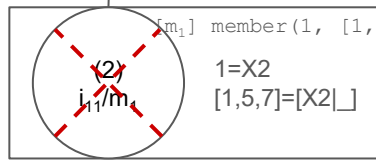
```
[q] ?- intersection([1,3,5], [1,5,7], Res).
Res = [1,5] ;
Res = [1]
```

However, node (4) is
lost to backtracking (in
favor to (4')), and as
such, the [5] is not a
part of the list

(1)
(3')
(4')
(5'')
 Res = [1|R1] = [1|R3] = [1|R4] = [1|[]] = [1]



We can see that
through node (1)
we receive [1] in
the list



```
[i1] intersection([HA|TA], B, [HA|R]) :-
[i11]      member(HA, B),
[i12]      intersection(TA, B, R).
[i2] intersection([HA|TA], B, R) :-
[i21]      intersection(TA, B, R).
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]      member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
Res = [1,5] ;
Res = [1]
```

Backtrack starts at (5''), tries to find another way to go but there is no i_4 to go to and it fails

Backtrack then goes to (4'), forces (4'') but it fails propagating the fail upwards until (2). The backtrack of node (2) forces (2'') from i_{11}/m_2 and this creates new nodes that fail themselves propagating the fail until (1') is created with q/i_2

Res = ⁽¹⁾[1|R1] = ^(3')[1|R3] = ^(4')[1|R4] = ^(5'')[1|[]] = [1]
; Next → backtrack

(1')
q/i₂
[1,3,5]=[HA1|TA1] → HA1=1, TA1=[3,5]
[1,5,7]=B1
Res=R1

(2')
i₂₁/i₂
[i₁₂] inter([3,5], [1,5,7], R1).
[3,5]=[HA3|TA3] → HA3=3, TA1=[5]
[1,5,7]=B1
R1=R3

(3)
i₂₁/i₁
[i₂₁] inter([5], [1,5,7], R3).
[5]=[HA4|TA4] → HA4=5, TA4=[]
[1,5,7]=B1
R3=[HA4|R4] → R3=[5|R4]

(4')
i₁₁/m₂
[m₂] member(5, [1,5,7]).
5=X5
[1,5,7]=[_ | T5] → T5=[5,7]

(5)
m₂₁/m₁
[m₂₁] member(5, [5,7]).
5=X6
[5,7]=[X6|_]

(6'')
i₁₂/i₃
[i₃] intersection([], [1,5,7], R4).
[]=[]
[1,5,7]=_
R4=[]

However, node (1') was lost to backtracking (in favor to (1')), and as such, the [1] is not a part of the list

```
[i1] intersection([HA|TA], B, [HA|R]) :-
[i11]     member(HA, B),
[i12]     intersection(TA, B, R).
[i2] intersection([HA|TA], B, R) :-
[i21]     intersection(TA, B, R).
[i3] intersection([], _, []).

[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]     member(X, T).

[q] ?- intersection([1,3,5], [1,5,7], Res).
```

Do notice that because the starting point is new (1') the rest of the tree is the same as the first answer

We can see that through node (4) we receive the [5] in the list

(1') (2') (3) (6'')

Res = R1 = R3= [5|R4] = [5|[]] = [5]

(1')
q/i₂
[1,3,5]=[HA1|TA1] → HA1=1, TA1=[3,5]
[1,5,7]=B1
Res=R1

(2')
i₂₁/i₂
[3,5]=[HA3|TA3] → HA3=3, TA1=[5]
[1,5,7]=B1
R1=R3

(3)
i₂₁/i₁
[5]=[HA4|TA4] → HA4=5, TA4=[]
[1,5,7]=B1
R3=[HA4|R4] → R3=[5|R4]

(4)
i₁₁/m₂
5=X5
[1,5,7]=[_ | T5] → T5=[5,7]

(5)
m₂₁/m₁
5=X6
[5,7]=[X6|_]

(6'')
i₁₂/i₃
[]=[]
[1,5,7]=_
R4=[]

```
[i1] intersection([HA|TA], B, [HA|R]) :-
[i11]     member(HA, B),
[i12]     intersection(TA, B, R).
[i2] intersection([HA|TA], B, R) :-
[i21]     intersection(TA, B, R).
[i3] intersection([], _, []).
```

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]     member(X, T).
```

```
[q] ?- intersection([1,3,5],[1,5,7], Res).
```

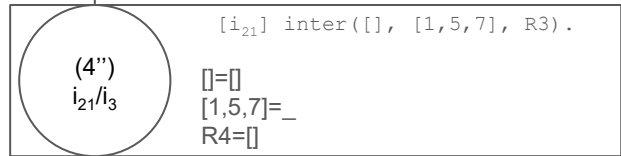
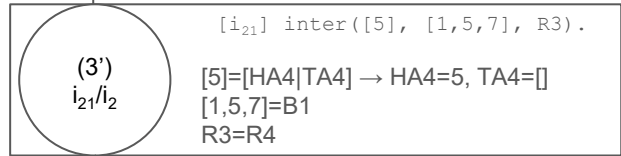
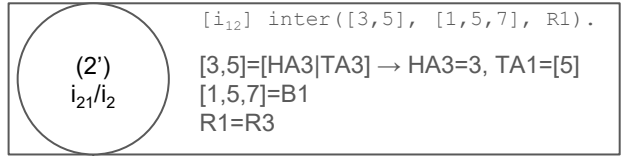
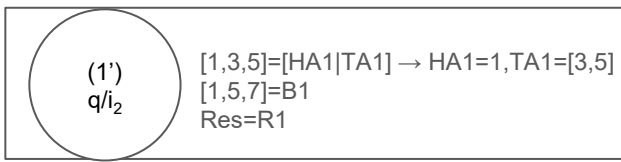
Backtrack starts at (6''), tries to find another way to go but there is no i₄ to go to and it fails

Backtrack then goes to (5), forces (5') from m₂₁/m₂ and this creates new nodes that fail themselves propagating the fail until (3') is created with i₂₁/i₂

We can see that through node (3) we receive the [5] in the list

(1') (2') (3) (6'')

Res = R1 = R3= [5|R4] = [5|[]] = [5]
; Next → backtrack



However, node (3) is lost to backtracking (in favor to (3')), and as such, the [5] is not a part of the list

```
[i1] intersection([HA|TA], B, [HA|R]) :-
[i11]         member(HA, B),
[i12]         intersection(TA, B, R).
[i2] intersection([HA|TA], B, R) :-
[i21]         intersection(TA, B, R).
[i3] intersection([], _, []).
```

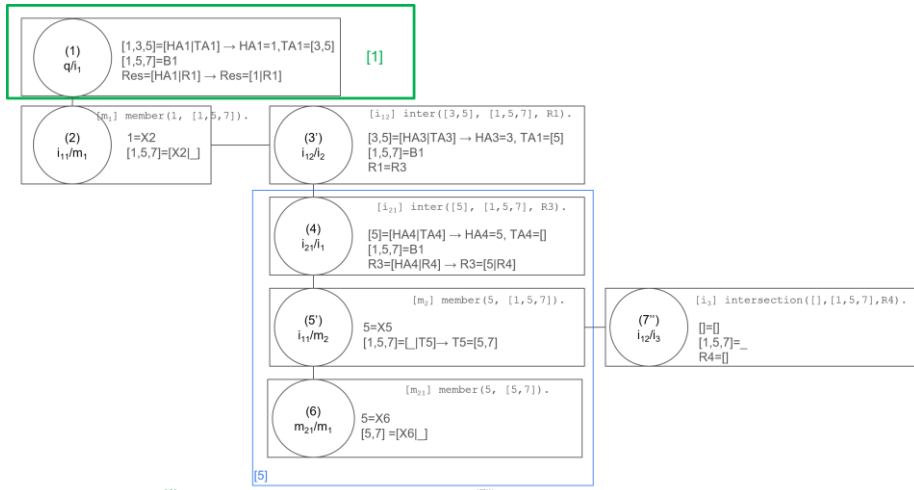
```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]         member(X, T).
```

```
[q] ?- intersection([1,3,5], [1,5,7], Res).
```

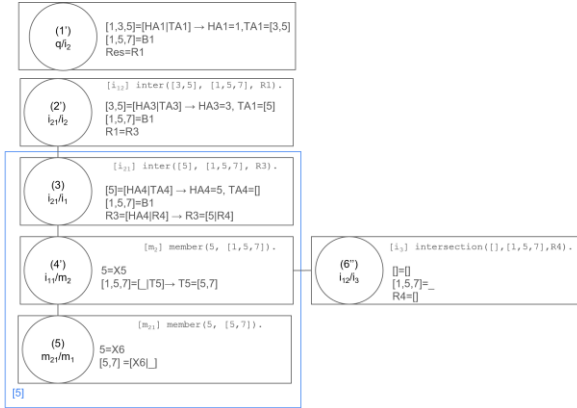
(1') (2') (3) (4'')

Res = R1 = R3= R4 = []

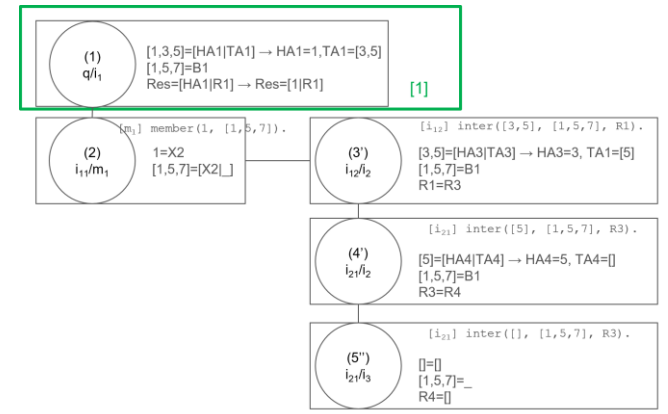
Last ; Next → backtrack, fails from (4'') and propagates until everything is unmade and the last answer is : false.



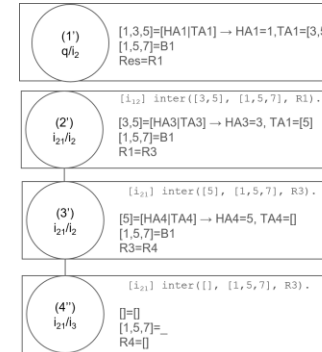
$$\text{Res} = [1|R1] = [1|R3] = [1|[5|R4]] = [1|[5|[]]] = [1,5]$$



$$\text{Res} = R1 = R3 = [5|R4] = [5|[]] = [5]$$



$$\text{Res} = [1|R1] = [1|R3] = [1|R4] = [1|[]] = [1]$$



$$\text{Res} = R1 = R3 = R4 = []$$